EMPLOYEE MANAGEMENT SYSTEM

WITH AI-BASED ANALYTICS

PROJECT REPORT

Submitted by

Jai Mukeshwaran V	**251801107**
Jaques Kevin R	**251801109**
Jothish Kumar P	**251801123**

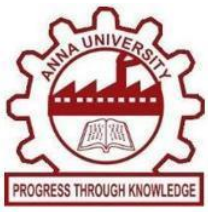
in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE





RAJALAKSHMI ENGINEERING COLLEGE
(AUTONOMOUS), CHENNAI - 602 105

NOV 2026

BONAFIDE CERTIFICATE

Certified that this Report titled “EMPLOYEE MANAGEMENT SYSTEM” is the bonafide work of “Jai Mukeshwaran V (251801107), Jaques Kevin R (251801109) and Jothish Kumar P (251801123)” who carried out the project work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Gnanasekar.J.M, M.E., Ph.D.,
Professor and Head
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College
Thandalam - 602 105

SIGNATURE

Mr. T. Rajesh, M.Tech.,
Assistant Professor (SS)
Department of Artificial Intelligence and Data Science
Rajalakshmi Engineering College
Thandalam - 602 105

Submitted to Project Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

STUDENT DECLARATION

We, **Jai Mukeshwaran V (251801107)**, **Jaques Kevin R (251801109)** and **Jothish Kumar P (251801123)**, students of B.Tech Artificial Intelligence and Data Science, Rajalakshmi Engineering College (Autonomous), Chennai, hereby declare that the project report titled **"Employee Management System"** is the result of our own work carried out under the guidance of Mr. T. Rajesh, M.Tech., Assistant Professor (SS), Department of Artificial Intelligence and Data Science.

We further declare that this work has not been submitted, in part or in full, to any other institution or university for the award of any degree, diploma or certificate. All sources of information, books, documentation and tools used in this report have been acknowledged in the References. Every rule, formula and code segment in this report is understood by the team members, and each member is able to explain the module he has developed.

Jai Mukeshwaran V (251801107)

Jaques Kevin R (251801109)

Place: Chennai
Date: ____ / ____ / 2026

Jothish Kumar P (251801123)

DEPARTMENT VISION AND MISSION

Vision

To become a global leader in Artificial Intelligence and Data Science by achieving through excellence in teaching, training, and research, to serve the society.

Mission

- To develop students' skills in innovation, problem-solving, and professionalism through the guidance of well-trained faculty.
- To encourage research activities among students and faculty members to address the evolving challenges of industry and society.
- To impart qualities such as moral and ethical values, along with a commitment to lifelong learning.

Programme Educational Objectives (PEOs)

- **PEO 1:** Build a successful professional career across industry, government, and academia by leveraging technology to develop innovative solutions for real-world problems.
- **PEO 2:** Maintain a learning mindset to continuously enhance knowledge through experience, formal education, and informal learning opportunities.
- **PEO 3:** Demonstrate an ethical attitude while excelling in communication, management, teamwork, and leadership skills.
- **PEO 4:** Utilize engineering, problem-solving, and critical thinking skills to drive social, economic, and sustainable impact.

Programme Specific Outcomes (PSOs)

- **PSO 1 Foundation Skills:** Apply the principles of AI and data science using problem-solving, inference, perception, knowledge representation and learning techniques.
- **PSO 2 Problem-Solving Skills:** Apply engineering principles and AI models to solve real-world problems across domains.
- **PSO 3 Successful Progression:** Use interdisciplinary knowledge to identify problems and develop solutions as an ethically responsible AI and data science professional.

Course Outcomes (COs)

- **CO 1:** Analyze and define a real-world problem by identifying key challenges, requirements and constraints.
- **CO 2:** Review existing solutions, identify research gaps and formulate research questions.
- **CO 3:** Develop a project plan with objectives, timelines and key deliverables.
- **CO 4:** Design and implement a prototype based on the proposed solution using appropriate AI tools and technologies.
- **CO 5:** Demonstrate teamwork, communication and project management skills by presenting a well-structured proposal and results.

PROBLEM STATEMENT

- An AI-powered system for intelligent employee management.
- Automates employee records, attendance, leave, and payroll.
- Uses AI analytics to monitor performance and optimize workloads.
- Enhances accuracy, productivity, and administrative efficiency.

ABSTRACT

Institutions depend on accurate employee records, attendance, leave and salary processing, yet many still maintain these in registers and spreadsheets. This causes calculation errors, delays, duplicate entries and no clear view of how staff are performing or how work is shared. This project presents an Employee Management System with AI-based analytics, developed in Java with Object-Oriented Programming and a MySQL database accessed through JDBC. Employees are modelled as classes (Person, Admin, Employee, TeachingStaff and NonTeachingStaff) together with Attendance, LeaveRequest, Task and PayrollSlip. The database has eight tables, is normalised to Third Normal Form, and enforces keys, UNIQUE and CHECK constraints, transactions, indexes and views. The Admin registers employees, marks attendance, approves leave, assigns tasks and generates monthly payroll; employees check in and out, apply for leave, view payslips and update tasks. The AI module computes a weighted performance score, classifies workload as underloaded, balanced or overloaded, suggests the best employee for a new task using a least-loaded heuristic, and raises attendance-risk alerts. The business logic was verified by 41 automated checks that all passed. The report also specifies a BCrypt password-hashing plan, anti-proxy attendance controls and a roadmap of AI enhancements.

Keywords: Employee management, Attendance, Leave management, Payroll, Performance analytics, Workload optimisation, Java, JDBC, MySQL, OOP, Normalisation, BCrypt

ACKNOWLEDGEMENT

We thank the Almighty for His blessings throughout this work. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E., F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and support in our premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal, for his kind support and the facilities provided to complete our work in time. We express our sincere thanks to Dr. J.M. GNANASEKAR, M.E., Ph.D., Professor and Head of the Department of Artificial Intelligence and Data Science, for his guidance and encouragement throughout the project work. We convey our deepest gratitude to our internal guide, Mr. T. RAJESH, M.Tech., Assistant Professor (SS), Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, for his valuable guidance throughout the project. We are also glad to thank our Project Coordinator, Mrs. D. MEENAKSHI M.E., Department of Artificial Intelligence and Data Science, for her useful tips during our reviews.

Jai Mukeshwaran V (251801107)
Jaques Kevin R (251801109)
Jothish Kumar P (251801123)

PROGRAMME OUTCOMES (PO)

- **PO1:** Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals and an engineering specialization to the solution of complex engineering problems.
- **PO2:** Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3:** Design / Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and the cultural, societal, and environmental considerations.
- **PO4:** Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of information to provide valid conclusions.
- **PO5:** Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6:** The engineer and society: Apply reasoning informed by contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7:** Environment and sustainability: Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
- **PO8:** Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.
- **PO9:** Individual and team work: Function effectively as an individual and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO10:** Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO11:** Project management and finance: Demonstrate knowledge and understanding of engineering management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO12:** Life-long learning: Recognize the need for and have the preparation and ability to engage in independent and lifelong learning in the broadest context of technological change.

COURSE OBJECTIVES AND CO-PO-PSO MAPPING

Course Objectives

- To identify and formulate real-world problems that can be solved using Artificial Intelligence and Data Science techniques.
- To apply theoretical and practical knowledge of AI & DS for designing innovative, data-driven solutions.
- To integrate various tools, frameworks, and algorithms to develop, test, and validate AI & DS models.
- To demonstrate effective teamwork, project management, and communication skills through collaborative project execution.
- To instill awareness of ethical, societal, and environmental considerations in the design and deployment of intelligent systems.

CO-PO-PSO Mapping

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
CO1	3	3	2	2	1	2	1	1	1	2	1	2	3	2	2
CO2	2	3	2	3	2	1	1	1	2	2	1	3	2	2	2
CO3	2	2	3	2	2	1	2	2	3	2	3	2	2	3	3
CO4	3	3	3	3	3	2	2	2	2	3	2	2	3	3	3
CO5	2	2	2	1	2	2	2	3	3	3	3	2	2	2	3

Correlation levels: 1 - Slight (Low), 2 - Moderate (Medium), 3 - Substantial (High). No correlation: “-”.

LIST OF ABBREVIATIONS

Abbreviation	Full form
OOP	Object-Oriented Programming
DBMS / RDBMS	(Relational) Database Management System
SQL	Structured Query Language
JDBC	Java Database Connectivity
JDK	Java Development Kit
IDE	Integrated Development Environment
PK / FK	Primary Key / Foreign Key
CRUD	Create, Read, Update, Delete
ACID	Atomicity, Consistency, Isolation, Durability
DFD	Data Flow Diagram
ER	Entity Relationship
HRMS	Human Resource Management System
HRA / DA	House Rent Allowance / Dearness Allowance
PF	Provident Fund
LOP	Loss of Pay
CL / ML / EL / OD	Casual / Medical / Earned Leave / On Duty
AI / ML	Artificial Intelligence / Machine Learning
GDPR	General Data Protection Regulation

TABLE OF CONTENTS

BONAFIDE CERTIFICATE **2**

STUDENT DECLARATION **3**

DEPARTMENT VISION AND MISSION **4**

PROBLEM STATEMENT **5**

ABSTRACT **5**

ACKNOWLEDGEMENT **6**

PROGRAMME OUTCOMES (PO) **7**

COURSE OBJECTIVES AND CO-PO-PSO MAPPING **8**

LIST OF ABBREVIATIONS **9**

TABLE OF CONTENTS **10**

CHAPTER 1INTRODUCTION **12**

1.1 Background 12

1.2 Problem Statement 12

1.3 Objectives 12

1.4 Compliance with the College Guidelines 12

1.5 Global Relevance of an Employee Management System 13

1.6 Preventing Proxy Attendance and Misuse 14

CHAPTER 2REQUIREMENTS AND FEASIBILITY **15**

2.1 Existing System and its Limitations 15

2.2 Proposed System 15

2.3 Functional Requirements 15

2.4 Non-Functional Requirements 15

2.5 Essential and Advanced Needs 16

2.6 Hardware and Software Requirements 16

2.7 Feasibility Study 16

CHAPTER 3SYSTEM ANALYSIS AND DESIGN **17**

3.1 Actors and Use Cases 17

3.2 Data Flow Diagrams 17

3.3 Architecture 18

3.4 Class Diagram 18

3.5 OOP Concepts Used 19

3.6 Sequence Diagrams 19

3.7 Business Rules 21

3.8 AI Analytics Module 22

3.9 Console Interface and Division of Work 22

CHAPTER 4DATABASE DESIGN **24**

4.1 Entity Relationship Diagram . . . 24

4.2 Tables and Keys . . . 24

4.3 Normalisation (1NF – 3NF) . . . 25

4.4 Transactions and ACID . . . 25

4.5 Indexes . . . 25

4.6 Views . . . 25

4.7 CRUD Mapping . . . 26

CHAPTER 5IMPLEMENTATION **27**

5.1 JDBC Connection . . . 27

5.2 Login with PreparedStatement and Polymorphic Mapping . . . 27

5.3 Transaction: Add Employee . . . 27

5.4 Attendance . . . 28

5.5 Leave: Overlap Check and Approval Transaction . . . 28

5.6 Payroll and AI Rules . . . 28

5.7 Input Validation . . . 29

CHAPTER 6VERIFICATION AND TEST PLAN **30**

6.1 Compilation and Execution . . . 30

6.2 Automated Verification Result . . . 30

6.3 Sample of the Actual Run . . . 30

6.4 Functional Test Cases . . . 30

CHAPTER 7SECURITY, LIMITATIONS AND MAINTENANCE **32**

7.1 Security Measures in the Current System . . . 32

7.2 The Plain-Text Password Problem . . . 32

7.3 BCrypt Hashing Plan . . . 32

7.4 Audit Trail . . . 32

7.5 Privacy and Data Protection . . . 33

7.6 Limitations . . . 33

7.7 Maintenance Plan . . . 33

CHAPTER 8CONCLUSION AND FUTURE ENHANCEMENTS **34**

8.1 Conclusion . . . 34

8.2 Future Enhancements . . . 34

REFERENCES **36**

APPENDIX ADATABASE SCHEMA **37**

APPENDIX BEXECUTION OUTPUT AND SCREENSHOTS **38**

APPENDIX CREQUIREMENT CHECKLIST **41**

CHAPTER 1
INTRODUCTION

1.1 Background

Every institution depends on its employees: teaching faculty, laboratory staff, office staff and administrators. Keeping their records, attendance, leave and salary correct is a daily task that consumes much time when done manually. This project develops an Employee Management System that stores all employee data in one MySQL database, automates attendance, leave and payroll rules, and applies explainable analytics to measure performance and balance workload. The system follows the Java + JDBC + MySQL flow expected for the project: user input to the Java program, to JDBC, to MySQL, and the retrieved records displayed through Java.

1.2 Problem Statement

- An AI-powered system for intelligent employee management.
- Automates employee records, attendance, leave, and payroll.
- Uses AI analytics to monitor performance and optimize workloads.
- Enhances accuracy, productivity, and administrative efficiency.

1.3 Objectives

- **Model the domain with OOP** using abstraction, inheritance, polymorphism and encapsulation.
- **Design a relational database** with keys, constraints, normalisation, indexes and views.
- **Manage employee records:** add, view, search, update and deactivate.
- **Automate attendance and leave** with balance checks and overlap prevention.
- **Automate payroll** with allowances, deductions and loss of pay.
- **Provide AI-based analytics:** performance score, workload analysis, task suggestion, attendance-risk alerts.
- **Provide a user-friendly console interface** with role-specific menus and plain-language messages.

1.3.1 Scope

In scope: a single-institution, console-based Java application with two roles (Admin, Employee) and MySQL persistence. Out of scope for this version: web or mobile front end, biometric devices, statutory income-tax computation and machine-learning models (these appear in Chapter 8).

1.4 Compliance with the College Guidelines

Table 2 Compliance with college guidelines

College condition	How this project satisfies it
Java application connected to MySQL using JDBC	Console application using MySQL Connector/J (Section 5.1)
Collect input from the user	Employee, attendance, leave, task and payroll inputs with validation (Section 5.7)
Insert records into MySQL	Employees, attendance, leave, tasks, payroll and reports are inserted (Chapter 5)
Retrieve and display records	Lists, reports and payslips read from MySQL and displayed in Java

College condition	How this project satisfies it
Apply OOP concepts	Abstraction, inheritance, polymorphism, encapsulation, exceptions (Section 3.5)
Start simple, then add features	Core CREATE, CONNECT, INSERT, RETRIEVE, DISPLAY first; then leave, payroll, AI
Team up to 4; do not copy	Three-member team with module ownership (Section 3.9); own design, all rules explained
Project title and 4-line description	Included (Section 1.2)
Chapter structure of the new template	Requirements and Feasibility; System Analysis and Design; Database Design; Implementation; Verification and Test Plan; Security, Limitations and Maintenance

1.5 Global Relevance of an Employee Management System

Employee administration is a universal need. Universities, schools, hospitals, factories, banks, government offices, retail chains and start-ups all have to answer the same questions: who works here, were they present, how much leave is left, how much must be paid, and is the work shared fairly. The design in this report is deliberately modular, so the same core (records, attendance, leave, payroll, tasks, analytics) can be re-configured for different organisations by changing constants and rules rather than rewriting the program.

Table 3 Where the system can be applied

Setting	How the system can be used	What must be configured
Colleges / schools	Teaching and non-teaching categories, timetable-linked workload, on-duty leave	Task-hour limits, academic allowance, leave quota
Hospitals	Shift rosters, night-duty allowance, overtime, emergency cover	Shift rules, overtime rates, leave-cover rules
Factories / retail	Shift attendance, overtime, absenteeism analytics	Shifts, overtime, weekly-off patterns
Government offices	Service records, leave rules, transparent approval trail	Multi-level approval, audit log
Remote / hybrid companies	Task-based workload balancing and performance dashboards	Web interface, time-zone handling
Small businesses / NGOs	Low-cost payroll and leave tracking without paid software	Simple rate table, backups

Country-specific rules

Payroll and leave law differ between countries, so the payroll rates in this project are constants that must be set according to local rules. Examples (to be verified against current law before real use): India has provident fund and state-level rules; the United Kingdom has statutory sick pay and tax withholding through PAYE; the United States has overtime and record-keeping rules under the Fair Labor Standards Act; the UAE requires salaries to be paid through its Wage Protection System; Singapore has the CPF contribution scheme; the European Union protects employee data under the GDPR. A deployable version would therefore load a country rule set (tax slabs, statutory contributions, working-hour limits, data-protection settings) instead of hard-coded rates.

Adoption is generally higher where payroll filing, digital payments and reliable connectivity are well established, and lower in regions with a large informal workforce or limited connectivity. In such regions a mobile-first, low-bandwidth and offline-tolerant design, with regional-language screens, is the practical way to extend the same system. This project does not measure country-wise adoption; the observation is a design consideration, not a research finding.

1.6 Preventing Proxy Attendance and Misuse

A common weakness of any attendance system is **proxy attendance**, where one person marks attendance for another. The system addresses this in layers: (1) every employee has a personal login and can mark only his or her own attendance; (2) the database allows one attendance row per employee per day (UNIQUE constraint), so a second entry cannot overwrite the first; (3) check-in time is taken from the server clock, not typed by the user; (4) manual marking by the Admin is recorded in an audit log with the Admin's id; and (5) the roadmap adds biometric/face verification, device binding and geofencing (Chapter 8). Similar controls apply to leave (ownership checks) and payroll (generated once per month, only by the Admin).

CHAPTER 2
REQUIREMENTS AND FEASIBILITY

2.1 Existing System and its Limitations

- **Register and spreadsheet records:** details, attendance and leave live in different files and become inconsistent.
- **Manual attendance totals:** counting present, late and absent days at month end is slow and error-prone.
- **Unchecked leave:** balances are tracked by hand, so excess or overlapping leave goes unnoticed.
- **Manual payroll:** allowances and loss-of-pay are calculated by hand, causing disputes.
- **No performance view:** there is no objective measure of attendance, punctuality and task completion.
- **Uneven workload:** tasks are assigned by habit; some staff are overloaded, others idle.

2.2 Proposed System

A Java and MySQL based system for two roles. The **Admin** adds, searches, updates and deactivates employees, marks attendance, approves or rejects leave, generates payroll, assigns tasks with an AI suggestion and views analytics. The **Employee** views the profile, checks in and out, applies for and cancels leave, views the balance and payslip, updates tasks and sees the personal performance score.

2.3 Functional Requirements

Table 4 Functional requirements

ID	Requirement	Role
FR1	Login by email and password; role-specific menu	Both
FR2	Add, view, search, update, deactivate employee	Admin
FR3	Daily check-in/out; late marking after 09:15	Employee
FR4	Mark absent; attendance report	Admin
FR5	Apply, view, cancel leave; balance check; overlap refusal	Employee
FR6	Approve or reject pending leave	Admin
FR7	Generate monthly payroll once per employee; view payslip	Admin / Employee
FR8	Assign and update tasks	Admin / Employee
FR9	Performance score, workload analysis, task suggestion, attendance-risk alerts	Admin (own score: Employee)

2.4 Non-Functional Requirements

- **Accuracy:** all amounts computed by the program, never typed.
- **Integrity:** rules enforced in Java and in MySQL (defence in depth).
- **Security:** parameterised SQL, ownership checks, password hashing plan (Chapter 7).
- **Usability:** numbered lists before choices; plain-language messages.
- **Maintainability:** layered architecture; rates and quotas are constants.
- **Portability:** runs on any system with JDK 8+ and MySQL.

2.5 Essential and Advanced Needs

Table 5 Essential and advanced needs

Need	Status
Employee master record, departments, role-based login	Included (plain-text password in demo; BCrypt planned)
Daily attendance with check-in/out and late marking	Included
Leave types, yearly quota and approval	Included
Monthly payroll with allowances, PF and loss of pay	Included
Task assignment and tracking, reports	Included
AI performance score, workload analysis, task suggestion, risk alerts	Included
Biometric / RFID / face attendance	Future
Machine-learning attrition and performance prediction	Future
Web / mobile interface, payslip PDF, e-mail/SMS alerts	Future
Multi-level approval	Future

2.6 Hardware and Software Requirements

Table 6 Hardware and software specification

Component	Specification
Processor	Intel Core i3 / AMD Ryzen 3 or above
RAM	4 GB (8 GB recommended)
Storage	1 GB free
Operating system	Windows 10 / 11
Language	Java (JDK 8 or above; tested with JDK 21)
Database	MySQL Server + MySQL Workbench
Driver	MySQL Connector/J
IDE	Visual Studio Code with Extension Pack for Java
Interface	Console

2.7 Feasibility Study

Table 7 Feasibility analysis

Type	Assessment
Technical	Java, JDBC and MySQL are mature, free and taught in the curriculum; no special hardware.
Economic	All tools are open source; zero licence cost compared with paid HRMS products.
Operational	Menu-driven with plain-language messages; staff need minimal training.
Schedule	Three modules developed in parallel by three members (Section 3.9).
Legal / ethical	Employee data is personal data: access limited by role; hashing and audit log planned.

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 Actors and Use Cases

Two actors interact with the system. The Admin manages staff and decisions; the Employee manages only personal data.

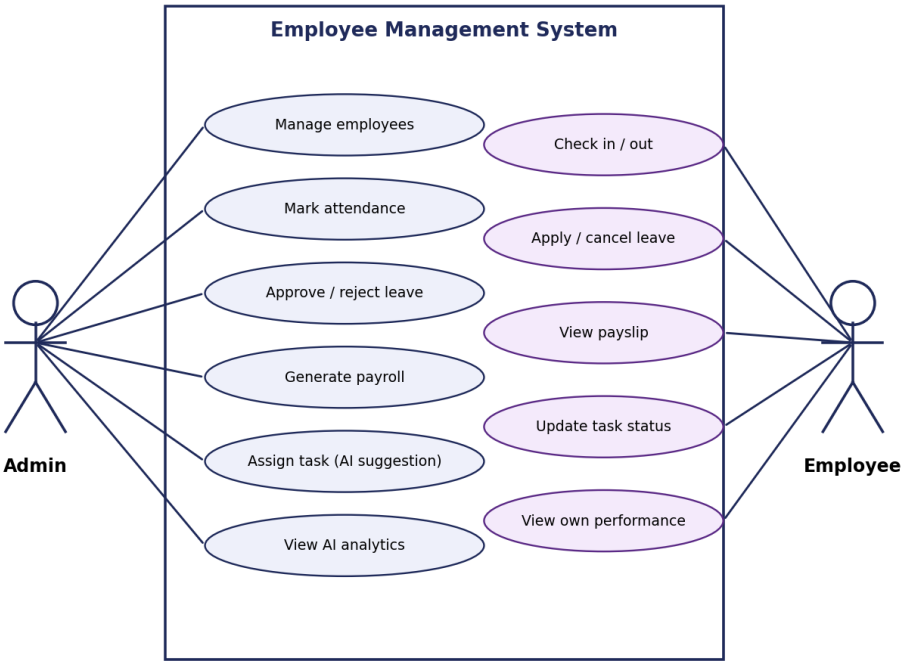


Fig 1 Use case diagram

3.2 Data Flow Diagrams

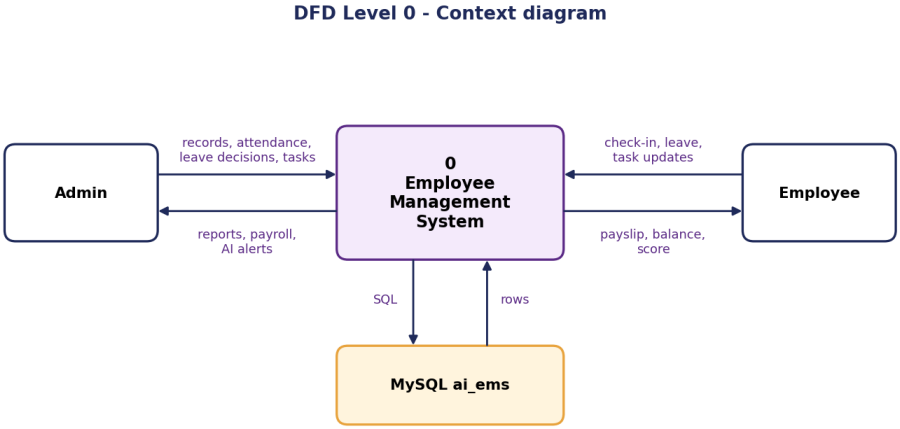


Fig 2 DFD Level 0 (context diagram)

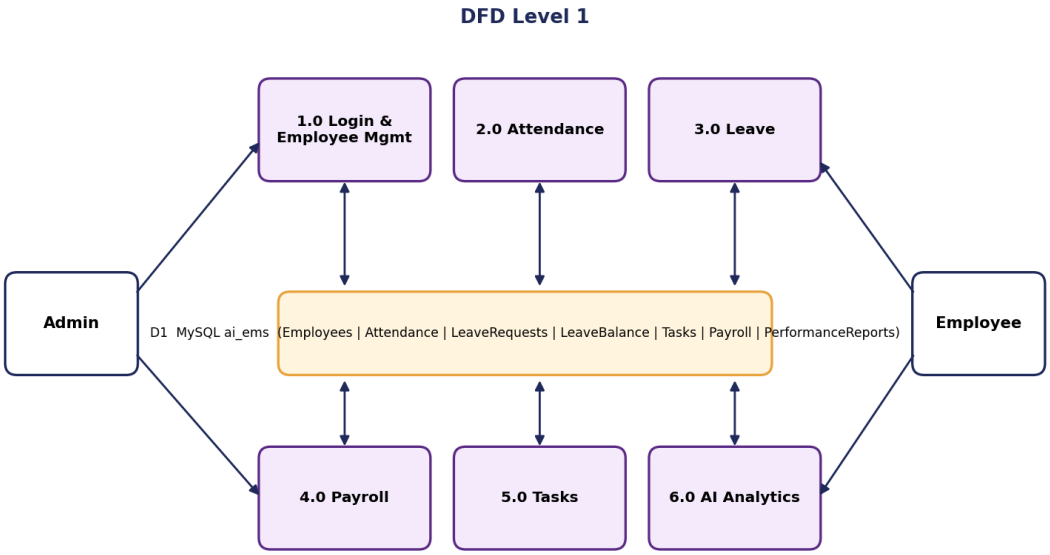


Fig 3 DFD Level 1 (main processes and the data store)

3.3 Architecture

The system is layered so that each class has one job. The presentation layer never writes SQL; the service layer contains every database operation and AI calculation and throws an AppException with a readable message when a rule is violated.

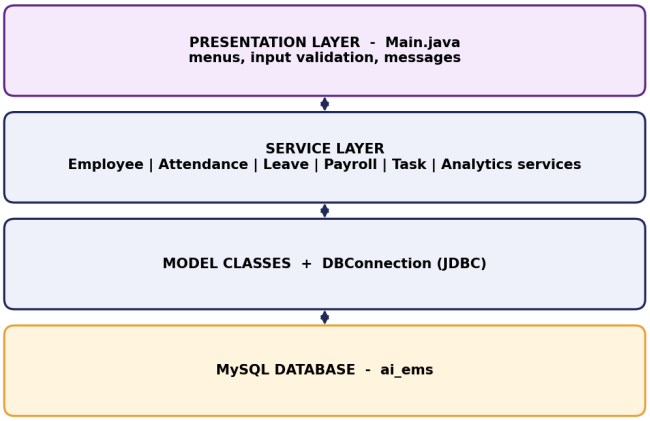


Fig 4 Layered architecture

3.4 Class Diagram

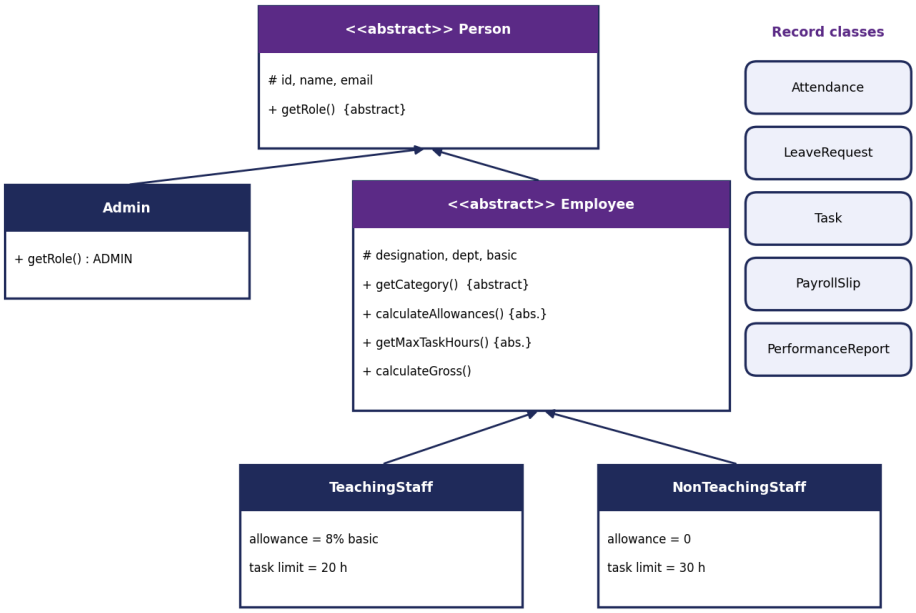


Fig 5 Class diagram of the model layer

3.5 OOP Concepts Used

Table 8 OOP concepts

Concept	Where it is used
Abstraction	Person and Employee are abstract classes that define what every user or employee must provide
Inheritance	Admin and Employee extend Person; TeachingStaff and NonTeachingStaff extend Employee
Polymorphism	calculateAllowances(), getMaxTaskHours(), getRole() behave differently in each subclass; payroll and workload code uses Employee references
Encapsulation	Fields are private/protected/final and accessed through getters
Exception handling	Rule violations raised as AppException and shown as readable messages

3.6 Sequence Diagrams

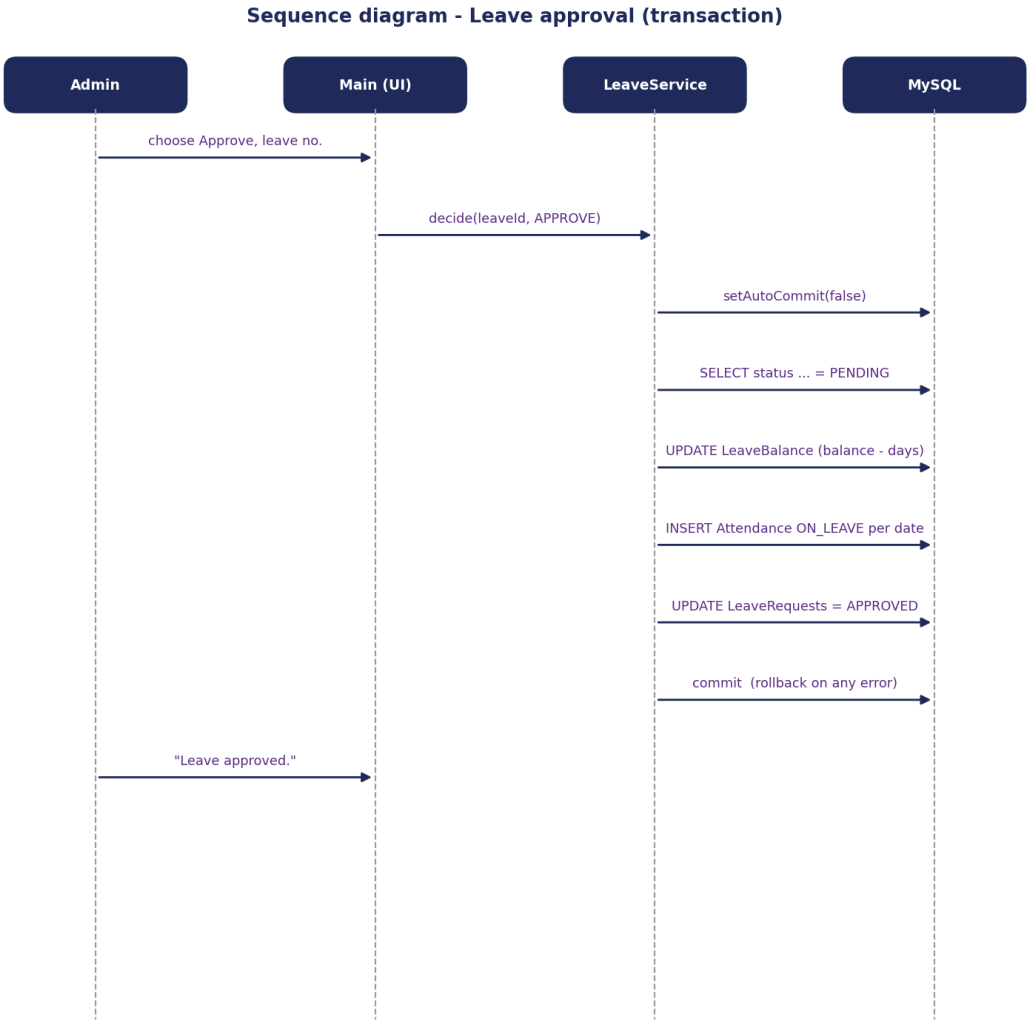


Fig 6 Sequence diagram: leave approval

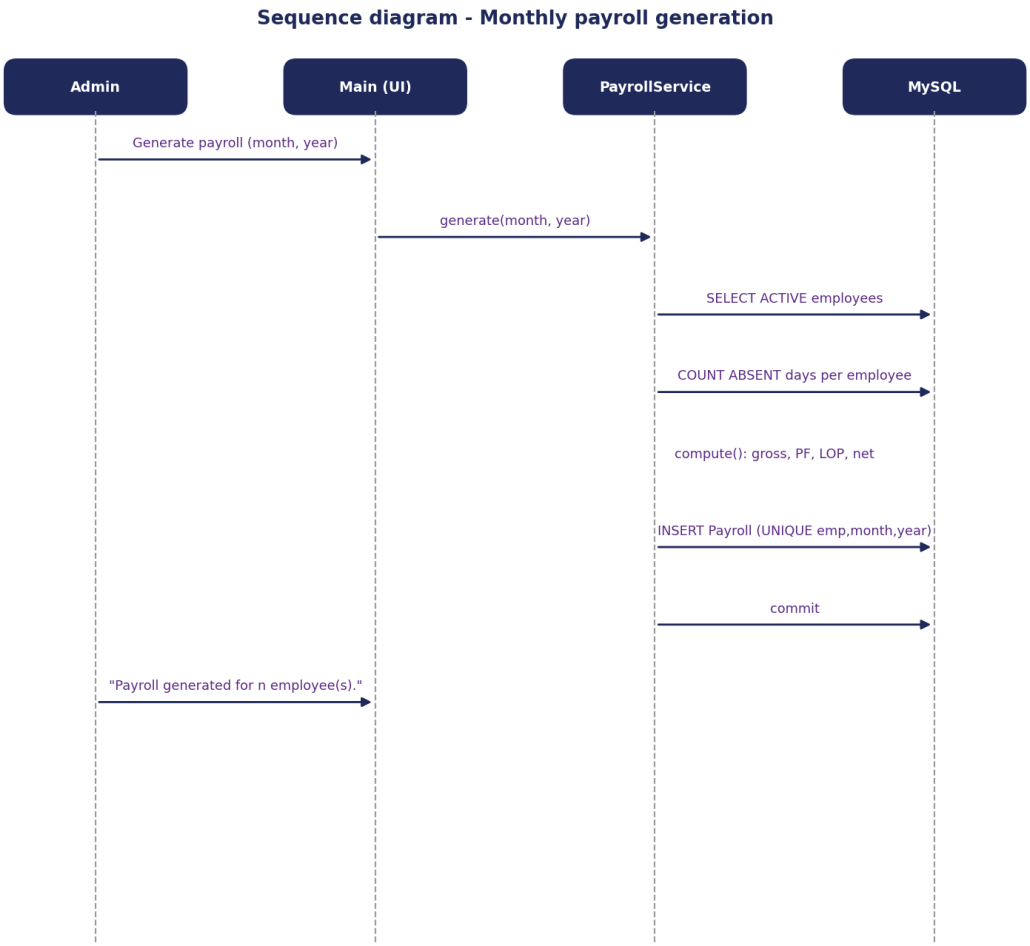


Fig 7 Sequence diagram: monthly payroll generation

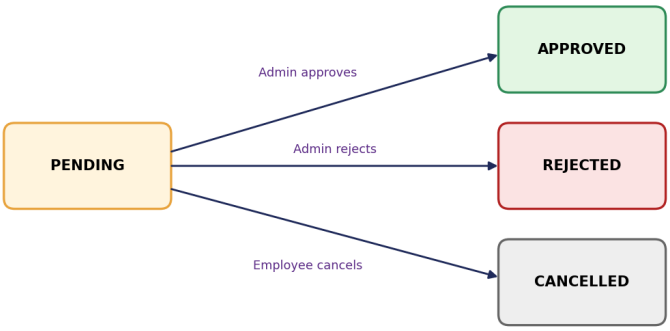


Fig 8 Leave status lifecycle

3.7 Business Rules

Attendance

- One check-in per day (UNIQUE emp_id + att_date). Check-in at or before 09:15 is PRESENT, after is LATE.
- Check-out only after check-in on the same day.
- The Admin may mark ABSENT. Approved leave days are stored as ON_LEAVE.

Leave

- Yearly quota (editable): CL 12, ML 10, EL 15, OD 10.
- Days = end date - start date + 1; start date not in the past.

- Refused if balance is insufficient or dates overlap another PENDING/APPROVED leave.
- Only PENDING requests can be approved, rejected or cancelled.

Payroll

Table 9 Payroll calculation rules

Component	Rule
HRA	10% of basic
DA	12% of basic
Academic allowance	8% of basic, teaching staff only
Gross	Basic + HRA + DA + academic allowance
PF	12% of basic
LOP days	Number of ABSENT days in the month
LOP amount	(Gross / 30) x LOP days
Net	Gross - PF - LOP amount

Worked example. Basic 50,000 (teaching): HRA 5,000, DA 6,000, allowance 4,000, gross 65,000; PF 6,000; with 2 absent days LOP = 65,000/30 x 2 = 4,333.33; net = 54,666.67. This value is reproduced by the program in Chapter 6.

3.8 AI Analytics Module

Four explainable, rule-based techniques are implemented in AnalyticsService and AnalyticsRules.

- **Performance score** = 0.4 x Attendance + 0.4 x Task + 0.2 x Punctuality. Rating: 85+ Excellent, 70–84 Good, 50–69 Average, below 50 Needs Improvement. Example: A=92, T=80, P=90 gives 86.8.
- **Workload analysis:** utilisation = pending hours / weekly limit (20 h teaching, 30 h non-teaching) x 100. Above 100% OVERLOADED; 40–100% BALANCED; below 40% UNDERLOADED.
- **Task suggestion (greedy least-loaded):** among ACTIVE employees of the department who are not on approved leave on the due date, pick the lowest utilisation after adding the new hours; ties go to the higher performance score.
- **Attendance risk:** LATE + ABSENT in the month: 5 or more HIGH, 3–4 MEDIUM, below 3 LOW.

These methods are intentionally simple so that every result can be traced to stored data. Why not machine learning now? With no historical data and a need for explainable HR decisions, transparent rules are the safer first step; Chapter 8 describes the ML upgrade path.

3.9 Console Interface and Division of Work

Admin menu: 1 Add employee, 2 View/search, 3 Update, 4 Deactivate/reactivate, 5 Mark attendance, 6 Attendance report, 7 Approve/reject leave, 8 Generate payroll, 9 Payroll report, 10 Assign task (AI suggestion), 11 Performance analytics, 12 Workload analysis, 13 Attendance-risk alerts, 0 Logout.

Employee menu: 1 Profile, 2 Check in/out, 3 My attendance, 4 Apply leave, 5 Leave status and balance, 6 Cancel pending leave, 7 My payslip, 8 My tasks, 9 Update task status, 10 My performance, 0 Logout.

Table 10 Division of work in the team

Member	Roll no.	Module
Jai Mukeshwaran V	251801107	Database design, DBConnection, EmployeeService, login
Jaques Kevin R	251801109	AttendanceService, LeaveService, PayrollService
Jothish Kumar P	251801123	TaskService, AnalyticsService, Main menus, testing

CHAPTER 4
DATABASE DESIGN

4.1 Entity Relationship Diagram

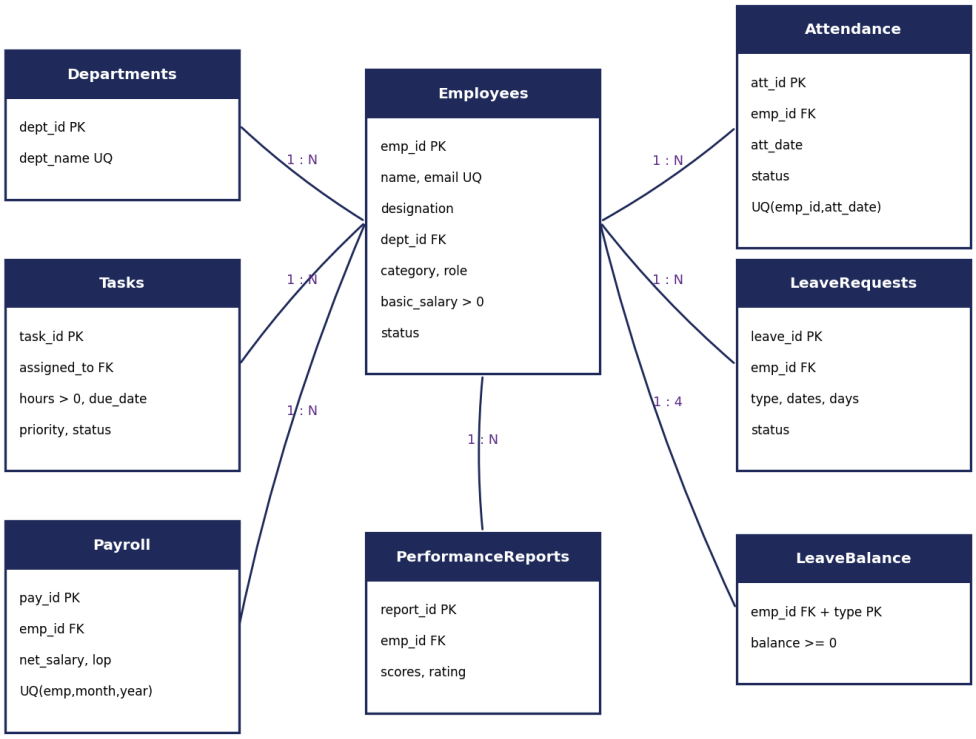


Fig 9 ER diagram of the ai_ems database

TeachingStaff and NonTeachingStaff are stored in the single Employees table; the column category decides which Java object is created when a row is read.

4.2 Tables and Keys

Table 11 Tables, keys and constraints

Table	Primary key	Foreign key / unique	Important constraints
Departments	dept_id	dept_name UNIQUE	NOT NULL name
Employees	emp_id	dept_id → Departments; email UNIQUE	basic_salary > 0; ENUM category, role, status
Attendance	att_id	emp_id → Employees; UNIQUE(emp_id, att_date)	ENUM status
LeaveBalance	(emp_id, leave_type)	emp_id → Employees	balance >= 0
LeaveRequests	leave_id	emp_id → Employees	end_date >= start_date; ENUM status
Tasks	task_id	assigned_to → Employees	hours > 0; ENUM priority, status
Payroll	pay_id	emp_id → Employees; UNIQUE(emp_id, month, year)	month 1–12
PerformanceReports	report_id	emp_id → Employees	scores NOT NULL

Key types used: primary keys (surrogate AUTO_INCREMENT ids; a composite key in LeaveBalance), foreign keys with ON DELETE CASCADE for dependent records,

candidate/alternate keys (email, dept_name) enforced with UNIQUE, and composite UNIQUE keys that express business rules (one attendance row per day, one payslip per month).

4.3 Normalisation (1NF – 3NF)

Consider a first, un-normalised design: one wide sheet Staff(emp_id, name, dept_name, dept_head, leave_cl, leave_ml, attendance_list, month_salary_list).

Table 12 Normalisation steps

Form	Problem found	Fix applied
1NF	Repeating groups: attendance_list and month_salary_list hold many values in one cell; leave_cl, leave_ml… are repeated columns	Move to Attendance, Payroll and LeaveBalance (one row per fact; atomic values)
2NF	In LeaveBalance(emp_id, leave_type, balance, leave_type_name) leave_type_name depends only on leave_type (partial dependency)	Keep only balance with the full key (emp_id, leave_type); type names are an ENUM
3NF	Employees(…, dept_id, dept_name) – dept_name depends on dept_id, not on emp_id (transitive dependency)	Departments table holds dept_name; Employees keeps only dept_id (FK)

The final eight tables satisfy 3NF: every non-key attribute depends on the key, the whole key and nothing but the key. Payroll stores computed amounts (hra, da, pf) deliberately: a payslip is a historical record and must not change if the rates are edited later. This is a documented, controlled denormalisation.

4.4 Transactions and ACID

Table 13 ACID in the project

Property	How it appears in this system
Atomicity	Add employee + 4 leave balances; approve leave (balance + attendance rows + status); payroll generation – all commit together or roll back together
Consistency	CHECK, UNIQUE and FOREIGN KEY constraints keep the database valid after every transaction (e.g. balance never negative)
Isolation	InnoDB isolation prevents two admins from approving the same request twice; the PENDING check is done inside the transaction
Durability	Committed data survives program or system restart (InnoDB redo log)

4.5 Indexes

Primary, UNIQUE and foreign-key columns are indexed automatically. Additional indexes support the most frequent queries (attendance by employee and date, pending leave, tasks by assignee).

```
CREATE INDEX idx_att_emp_date   ON Attendance(emp_id, att_date);
CREATE INDEX idx_leave_emp      ON LeaveRequests(emp_id, status);
CREATE INDEX idx_tasks_assignee ON Tasks(assigned_to, status);
CREATE INDEX idx_emp_dept       ON Employees(dept_id, status);
```

Additional indexes

4.6 Views

Views hide the aggregate queries used by the analytics module and give the Java code a simple table to read.

```
CREATE VIEW v_monthly_attendance AS
SELECT emp_id, YEAR(att_date) yr, MONTH(att_date) mon,
       SUM(status='PRESENT') present, SUM(status='LATE') late,
       SUM(status='ABSENT') absent, SUM(status='ON_LEAVE') on_leave
FROM Attendance GROUP BY emp_id, YEAR(att_date), MONTH(att_date);
```

```
CREATE VIEW v_pending_load AS
SELECT assigned_to emp_id, SUM(hours) pending_hours
FROM Tasks WHERE status IN ('PENDING','IN_PROGRESS') GROUP BY assigned_to;
```

Views for monthly attendance and pending workload

4.7 CRUD Mapping

Table 14 CRUD mapping of console menus

Menu option	Role	C / R / U / D	SQL operation (tables)
Add employee	Admin	C	INSERT Employees; INSERT LeaveBalance x4 (transaction)
View / search employees	Admin	R	SELECT Employees JOIN Departments
Update employee	Admin	U	UPDATE Employees
Deactivate / reactivate	Admin	U	UPDATE Employees SET status (soft delete)
Mark attendance	Admin	C	INSERT Attendance (ABSENT)
Attendance report	Admin	R	SELECT Attendance / view v_monthly_attendance
Approve / reject leave	Admin	U + C	UPDATE LeaveRequests, LeaveBalance; INSERT Attendance (transaction)
Generate payroll	Admin	C	INSERT Payroll (UNIQUE guard)
Payroll report	Admin	R	SELECT Payroll JOIN Employees
Assign task	Admin	C	INSERT Tasks; SELECT workload view
Performance analytics	Admin	C + R	INSERT PerformanceReports; SELECT aggregates
Workload / risk reports	Admin	R	SELECT v_pending_load, v_monthly_attendance
Login	Both	R	SELECT Employees WHERE email AND password
Check in / check out	Employee	C / U	INSERT / UPDATE Attendance
Apply leave	Employee	C	SELECT overlap check; INSERT LeaveRequests
Cancel leave	Employee	U	UPDATE LeaveRequests SET CANCELLED
Leave balance, payslip, tasks	Employee	R	SELECT LeaveBalance / Payroll / Tasks (own emp_id only)
Update task status	Employee	U	UPDATE Tasks

Delete policy: employees are never physically deleted from the menu; they are set to INACTIVE so that attendance, leave and payroll history remain intact. Leave is cancelled, not deleted.

CHAPTER 5
IMPLEMENTATION

This chapter shows the central code of each module. The snippets follow the design of Chapters 3 and 4; the Java logic of the OOP model, payroll and AI rules was compiled and executed for verification (Chapter 6). Database-access snippets require MySQL and are executed on the project computer.

5.1 JDBC Connection

```
public class DBConnection {
    private static final String URL  = "jdbc:mysql://localhost:3306/ai_ems";
    private static final String USER = "root";
    private static final String PASS = "your_password";   // move to config file later

    public static Connection get() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

DBConnection.java

5.2 Login with PreparedStatement and Polymorphic Mapping

```
public Employee login(String email, String password) throws AppException {
    String sql = "SELECT * FROM Employees WHERE email = ? AND password = ? "
               + "AND status = 'ACTIVE'";
    try (Connection c = DBConnection.get();
         PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setString(1, email);          // ? placeholders -> no SQL injection
        ps.setString(2, password);       // demo only: BCrypt in Chapter 7
        try (ResultSet rs = ps.executeQuery()) {
            if (!rs.next()) throw new AppException("Wrong email or password.");
            return map(rs);              // builds TeachingStaff / NonTeachingStaff
        }
    } catch (SQLException e) {
        throw new AppException("Database problem. Please try again.");
    }
}
```

EmployeeService.login()

```
private Employee map(ResultSet rs) throws SQLException {
    String cat = rs.getString("category");
    int id = rs.getInt("emp_id");
    String name = rs.getString("name"), des = rs.getString("designation");
    double basic = rs.getDouble("basic_salary");
    if (cat.equals("TEACHING"))
        return new TeachingStaff(id, name, des, rs.getInt("dept_id"), basic);
    return new NonTeachingStaff(id, name, des, rs.getInt("dept_id"), basic);
}
```

Mapping a row to TeachingStaff or NonTeachingStaff

5.3 Transaction: Add Employee

```
public int addEmployee(Employee e, String email, String pwd) throws AppException {
    Connection c = null;
    try {
        c = DBConnection.get();
        c.setAutoCommit(false);                          // start transaction
        PreparedStatement ps = c.prepareStatement(
            "INSERT INTO Employees(name,email,password,designation,dept_id,"
          + "category,role,join_date,basic_salary) VALUES (?,?,?,?,?,?,?,?,?)",
            Statement.RETURN_GENERATED_KEYS);
        // ... set the 9 parameters ...
        ps.executeUpdate();
        ResultSet keys = ps.getGeneratedKeys(); keys.next();
        int empId = keys.getInt(1);
        String[] types = {"CL", "ML", "EL", "OD"};
        int[]    quota = {12, 10, 15, 10};
        PreparedStatement lb = c.prepareStatement(
            "INSERT INTO LeaveBalance(emp_id,leave_type,balance) VALUES (?,?,?)");
        for (int i = 0; i < 4; i++) {
```

```
            lb.setInt(1, empId); lb.setString(2, types[i]); lb.setInt(3, quota[i]);
            lb.addBatch();
        }
        lb.executeBatch();
        c.commit();                                     // all-or-nothing
        return empId;
    } catch (SQLIntegrityConstraintViolationException dup) {
        rollback(c); throw new AppException("This email is already registered.");
    } catch (SQLException ex) {
        rollback(c); throw new AppException("Could not add employee.");
    }
}
```

EmployeeService.addEmployee() - commit / rollback

5.4 Attendance

```
public String checkIn(int empId) throws AppException {
    LocalTime now = LocalTime.now();
    String status = now.isAfter(LocalTime.of(9, 15)) ? "LATE" : "PRESENT";
    String sql = "INSERT INTO Attendance(emp_id,att_date,check_in,status) "
               + "VALUES (?, CURDATE(), ?, ?)";
    try (Connection c = DBConnection.get(); PreparedStatement ps = c.prepareStatement(sql)) {
        ps.setInt(1, empId); ps.setTime(2, Time.valueOf(now)); ps.setString(3, status);
        ps.executeUpdate();
        return status;
    } catch (SQLIntegrityConstraintViolationException e) {  // UNIQUE(emp_id, att_date)
        throw new AppException("Attendance for today is already recorded.");
    } catch (SQLException e) { throw new AppException("Database problem."); }
}
```

AttendanceService.checkIn() - late rule and UNIQUE guard

5.5 Leave: Overlap Check and Approval Transaction

```
String sql = "SELECT COUNT(*) FROM LeaveRequests WHERE emp_id = ? "
           + "AND status IN ('PENDING','APPROVED') "
           + "AND start_date <= ? AND end_date >= ?";
ps.setInt(1, empId);
ps.setDate(2, Date.valueOf(end));      // requested end date
ps.setDate(3, Date.valueOf(start));    // requested start date
ResultSet rs = ps.executeQuery(); rs.next();
if (rs.getInt(1) > 0)
    throw new AppException("You already have leave on some of these dates.");
```

Overlap query in LeaveService.applyLeave()

```
public void decide(int leaveId, boolean approve) throws AppException {
    Connection c = null;
    try {
        c = DBConnection.get();
        c.setAutoCommit(false);
        LeaveRequest r = load(c, leaveId);               // SELECT ... WHERE leave_id=?
        if (!r.getStatus().equals("PENDING"))
            throw new AppException("Only PENDING leave can be approved or rejected.");
        if (approve) {
            update(c, "UPDATE LeaveBalance SET balance = balance - ? "
                    + "WHERE emp_id = ? AND leave_type = ?", r.days, r.empId, r.type);
            for (LocalDate d = r.start; !d.isAfter(r.end); d = d.plusDays(1))
                update(c, "INSERT INTO Attendance(emp_id,att_date,status) "
                        + "VALUES (?,?,'ON_LEAVE')", r.empId, d);
        }
        update(c, "UPDATE LeaveRequests SET status = ? WHERE leave_id = ?",
               approve ? "APPROVED" : "REJECTED", leaveId);
        c.commit();
    } catch (SQLException ex) { rollback(c); throw new AppException("Leave not updated."); }
}
```

LeaveService.decide() - one transaction over three tables

5.6 Payroll and AI Rules

```
public static PayrollSlip compute(Employee e, int lopDays) {
    double basic = e.getBasic();
    double hra = basic * 0.10, da = basic * 0.12;
    double allow = e.calculateAllowances();          // polymorphic: 8% or 0
    double gross = basic + hra + da + allow;
    double pf  = basic * 0.12;
    double lop = Math.round(gross / 30 * lopDays * 100.0) / 100.0;
```

```
    return new PayrollSlip(e.getId(), basic, hra, da, allow, pf, lopDays, lop,
                           gross - pf - lop);
}
```

PayrollService.compute() - polymorphic salary calculation

```
static double overall(double a, double t, double p) { return 0.4*a + 0.4*t + 0.2*p; }
static String rating(double s) {
    return s >= 85 ? "Excellent" : s >= 70 ? "Good" : s >= 50 ? "Average"
                                                            : "Needs Improvement";
}
static String workload(double pendingHrs, int maxHrs) {
    double u = pendingHrs / maxHrs * 100;
    return u > 100 ? "OVERLOADED" : u >= 40 ? "BALANCED" : "UNDERLOADED";
}
static String risk(int lateAbsent) {
    return lateAbsent >= 5 ? "HIGH" : lateAbsent >= 3 ? "MEDIUM" : "LOW";
}
```

AnalyticsRules - scoring, rating, workload, risk

```
// Greedy least-loaded heuristic
candidates.sort(Comparator
    .comparingDouble((Candidate c) -> c.loadAfter(newHours))   // lowest load first
    .thenComparing(c -> -c.performanceScore));                 // tie -> higher score
Candidate best = candidates.get(0);
```

Task-assignment suggestion

5.7 Input Validation

```
static int readInt(Scanner in, String prompt) {
    while (true) {
        System.out.print(prompt);
        try { return Integer.parseInt(in.nextLine().trim()); }
        catch (NumberFormatException e) { System.out.println("Please enter a whole number."); }
    }
}
static LocalDate readDate(Scanner in, String prompt) {
    while (true) {
        System.out.print(prompt);
        try { return LocalDate.parse(in.nextLine().trim()); }
        catch (DateTimeParseException e) {
            System.out.println("Please use the format YYYY-MM-DD, e.g. 2026-11-05.");
        }
    }
}
```

Helper methods in Main.java

CHAPTER 6
VERIFICATION AND TEST PLAN

6.1 Compilation and Execution

The logic classes (Person, Admin, Employee, TeachingStaff, NonTeachingStaff, payroll computation and analytics rules) and the self-check program were executed with JDK 21 using `java EmsLogic.java`. Database-dependent features are tested on the project computer with MySQL (Section 6.4). To run the full application on Windows:

```
javac -cp ".;mysql-connector-j-<version>.jar" *.java
java  -cp ".;mysql-connector-j-<version>.jar" Main
```

6.2 Automated Verification Result

41 automated checks covering OOP/polymorphism, payroll, performance score and ratings, workload levels, attendance risk, task-suggestion ranking, leave-day counting, overlap detection and the 09:15 late rule were run. **Result: 41 passed, 0 failed.** The full console output is reproduced in Appendix B.

6.3 Sample of the Actual Run

```
=== 1. OOP and polymorphism ===
Arun Kumar -> TEACHING, allowance 4000.00, task limit 20 h
Meena Devi -> NON_TEACHING, allowance 0.00, task limit 30 h
[PASS] Teaching task limit  expected=20  actual=20
[PASS] Non-teaching task limit  expected=30  actual=30
[PASS] Admin role  expected=ADMIN  actual=ADMIN

=== 2. Payroll calculation ===
-------------------------------------------
PAYSLIP 10/2026
Employee : Arun Kumar (No. 2)
Basic : 50000.00
HRA (10%) : 5000.00
DA (12%) : 6000.00
Academic allow. : 4000.00
Gross salary : 65000.00
PF (12%) : 6000.00
LOP (2 day(s)) : 4333.33
NET SALARY : 54666.67
-------------------------------------------
[PASS] Teaching gross  expected=65000.00  actual=65000.00
[PASS] Teaching PF  expected=6000.00  actual=6000.00
```

First part of the console output (payroll example reproduced exactly)

6.4 Functional Test Cases

Table 15 Functional test cases and current status

ID	Scenario	Expected output	Status
TC01	Admin adds teaching employee	Employee added; 4 leave balances	Run on MySQL – result to be filled
TC02	Add with existing email	“This email is already registered.”	Run on MySQL – result to be filled
TC03	Add with salary 0	“The amount must be greater than 0.”	Run on MySQL – result to be filled
TC04	Login with wrong password	“Wrong email or password.”	Run on MySQL – result to be filled
TC05	Check in at/before 09:15	PRESENT	Verified (logic run)
TC06	Check in after 09:15	LATE	Verified (logic run)
TC07	Second check-in same day	“Attendance for today is already recorded.”	Run on MySQL – result to be filled

ID	Scenario	Expected output	Status
TC08	Check out without check-in	“Please check in first.”	Run on MySQL – result to be filled
TC09	Apply casual leave within balance	Leave PENDING; days counted	Verified (logic run) (day count)
TC10	Leave start date in the past	“Start date cannot be in the past.”	Run on MySQL – result to be filled
TC11	Leave exceeds balance	“You have only 12 days of Casual Leave left.”	Run on MySQL – result to be filled
TC12	Overlapping leave	“You already have leave on some of these dates.”	Verified (logic run) (overlap rule)
TC13	Admin approves leave	Balance reduced; ON_LEAVE rows	Run on MySQL – result to be filled
TC14	Approve non-pending leave	“Only PENDING leave can be approved or rejected.”	Run on MySQL – result to be filled
TC15	Employee cancels pending leave	“Leave cancelled.”	Run on MySQL – result to be filled
TC16	Generate payroll	“Payroll generated for n employee(s).”	Run on MySQL – result to be filled
TC17	Generate payroll twice	“Payroll for this month is already generated.”	Run on MySQL – result to be filled
TC18	Payroll with 2 absent days	LOP 4,333.33; net 54,666.67	Verified (logic run)
TC19	View another employee's payslip	Not possible from menu	Run on MySQL – result to be filled
TC20	Assign task with AI suggestion	Least-loaded employee first	Verified (logic run) (ranking)
TC21	Task for employee on leave	Skipped with message	Run on MySQL – result to be filled
TC22	Update task status	“Task updated.”; date stored	Run on MySQL – result to be filled
TC23	Performance report	A, T, P, overall, rating	Verified (logic run)
TC24	Workload report	OVERLOADED / BALANCED / UNDERLOADED	Verified (logic run)
TC25	Invalid date format	Asked again with example	Run on MySQL – result to be filled

Cases marked “Run on MySQL” must be executed and the observed output and screenshot added in Appendix B before submission; no result has been invented for them.

CHAPTER 7
SECURITY, LIMITATIONS AND MAINTENANCE

7.1 Security Measures in the Current System

- **SQL injection:** every statement with user input is a PreparedStatement.
- **Role-based access:** Admin and Employee menus are separate; employee queries always use the logged-in emp_id (ownership check).
- **Database integrity:** constraints prevent invalid data even if Java code has a defect.
- **Safe messages:** users see plain-language messages, never raw SQL errors.

7.2 The Plain-Text Password Problem

In the demonstration version passwords are stored as plain text in the Employees table. This is unsafe: anyone with database access (or a leaked backup) can read every password, and because people reuse passwords, the damage spreads beyond this system. A real system must never store or e-mail a password; it must store only a salted, slow hash.

7.3 BCrypt Hashing Plan

Table 16 BCrypt migration plan

Step	Action
1	Add library jBCrypt (or Spring Security Crypto) to the classpath.
2	Alter table: rename column password to password_hash and widen it to VARCHAR(100) (a BCrypt hash is 60 characters).
3	On registration and password change: hash with BCrypt.hashpw(plain, BCrypt.gensalt(12)); store only the hash.
4	On login: fetch the hash by email and verify with BCrypt.checkpw; never compare plain text.
5	Migration: existing demo users are forced to set a new password at first login (old plain values are overwritten).
6	Add rules: minimum length, lock after repeated failed attempts, and write login failures to AuditLog.
7	Move DB user and password out of source code into a configuration file or environment variable.

```
// Registration: store only the hash
String hash = BCrypt.hashpw(plainPassword, BCrypt.gensalt(12));
// INSERT INTO Employees(..., password_hash) VALUES (..., ?)

// Login: fetch hash by email, then compare
String stored = rs.getString("password_hash");
if (!BCrypt.checkpw(enteredPassword, stored))
    throw new AppException("Wrong email or password.");
```

BCrypt registration and login check

BCrypt adds a random salt to each password and is deliberately slow (the cost factor, 12 here, doubles the work for each increase), so stolen hashes are very expensive to crack and identical passwords produce different hashes.

7.4 Audit Trail

An AuditLog table records who did what and when for sensitive actions: login failures, manual attendance, leave decisions, payroll generation and employee deactivation.

```
CREATE TABLE AuditLog (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  actor_id INT NOT NULL, action VARCHAR(60) NOT NULL,
  detail VARCHAR(255), logged_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (actor_id) REFERENCES Employees(emp_id)
```

```
);
```

AuditLog table

7.5 Privacy and Data Protection

Employee records contain personal and salary data. The system should collect only what is needed, restrict viewing by role, avoid logging sensitive values, and follow the data-protection law of the country of use (for example the GDPR in the European Union). Backups must be encrypted and access to them restricted.

7.6 Limitations

- Console interface only; not suitable for non-technical staff without a GUI/web front end.
- Passwords are plain text in the demo (Section 7.2).
- AI techniques are rule-based; weights, thresholds and quotas are assumptions to be set by college regulations.
- Attendance is by check-in/out or Admin entry; no biometric device.
- Payroll is simplified (30-day month; no income tax, bonuses or arrears).
- Single-level approval by the Admin.
- Database credentials are in DBConnection.java.

7.7 Maintenance Plan

Table 17 Maintenance plan

Type	Activity	Frequency
Corrective	Fix defects found in use; add regression test	As reported
Adaptive	Update rates, quotas and cut-off times when college rules change (constants)	Each academic year
Perfective	New reports, GUI, notifications	Per release
Preventive	Backups (mysqldump), restore test, dependency updates, index review	Daily / monthly
Security	Review audit log, rotate DB credentials, patch MySQL and JDK	Monthly

CHAPTER 8
CONCLUSION AND FUTURE ENHANCEMENTS

8.1 Conclusion

This project presents an Employee Management System that manages employee records, attendance, leave and payroll through a Java application and a MySQL database. OOP models people and employee categories; a normalised relational database with keys, constraints, transactions, indexes and views stores the data reliably. The AI module calculates performance scores, detects overloaded and underloaded employees, suggests the best employee for a task and warns about attendance risk. Automatic calculation of leave and salary reduces manual work and errors, and data-based reports support fair decisions. The business logic was verified by 41 passing automated checks, and the report documents analysis, design, implementation, testing, security and maintenance as required by the college template.

8.2 Future Enhancements

A. AI and machine-learning enhancements

Table 18 AI roadmap

Enhancement	What it does	Data / technique
Attrition prediction	Predicts which employees are likely to leave so HR can act early	Tenure, leave pattern, scores, salary band; logistic regression / gradient boosting; explain with feature importance
HR chatbot	Answers “How many CL days do I have?”, “Show my payslip” in natural language	Intent classification + the existing service layer; regional-language support
Resume screening	Ranks applicants against a job description	Text embeddings + similarity; human makes the final decision; bias audit mandatory
Smart scheduling	Builds timetables, duty rosters and shifts that respect leave and workload	Constraint programming / optimisation replacing the greedy heuristic
Anomaly detection	Flags unusual check-in patterns, suspected proxy attendance, payroll outliers	Isolation Forest / z-score on attendance and payroll history
Payroll forecasting	Forecasts monthly salary cost and leave liability	Time-series models (ARIMA / Prophet) on Payroll history
Performance prediction	Early warning when a score is trending down	Regression on monthly PerformanceReports
Sentiment / feedback analysis	Summarises employee feedback and surveys	NLP classification; anonymised and aggregated

Responsible-AI note: HR decisions affect livelihoods. Any ML model must be explainable, tested for bias across groups, used only as decision support, and subject to human review and the employee's right to question a result.

B. System enhancements

- Biometric, RFID or face-recognition attendance with liveness check; device binding and geofencing to stop proxy attendance.
- Web and mobile interface with regional-language support and offline-tolerant mobile check-in.
- Secure authentication (BCrypt, multi-factor) and credentials in a configuration file.
- Multi-level approval: Department Head, then Admin.
- Payslip PDF and e-mail/SMS notifications.

- Statutory payroll by country: income tax, social-security contributions, bonuses, arrears, bank-transfer files.
- Institution modules: timetable-based faculty workload, duty rosters, on-duty and permission requests.
- Dashboards: department-wise attendance, payroll cost and performance trends.
- Code quality: DAO pattern, connection pooling, JUnit tests and continuous integration.

REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Pearson, 2016.

[3] H. Schildt, Java: The Complete Reference, 12th ed. McGraw-Hill Education (Oracle Press), 2021.

[4] Oracle, “JDBC Basics,” The Java Tutorials. https://docs.oracle.com/javase/tutorial/jdbc/basics/

[5] Oracle, “MySQL Connector/J Developer Guide” and “MySQL Reference Manual.” https://dev.mysql.com/doc/

[6] Zoho Corporation, “Zoho People: HR management software,” company website.

[7] Greytip Software, “greytHR: HR and payroll software,” company website.

[8] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

[9] J. Han, M. Kamber, and J. Pei, Data Mining: Concepts and Techniques, 3rd ed. Morgan Kaufmann, 2011.

[10] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” USENIX Annual Technical Conference, 1999 (BCrypt).

[11] Regulation (EU) 2016/679 (General Data Protection Regulation).

APPENDIX A
DATABASE SCHEMA

```
DROP DATABASE IF EXISTS ai_ems;  CREATE DATABASE ai_ems;  USE ai_ems;
CREATE TABLE Departments (dept_id INT AUTO_INCREMENT PRIMARY KEY, dept_name VARCHAR(100) NOT NULL UNIQUE);
CREATE TABLE Employees (
 emp_id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100) NOT NULL, email VARCHAR(120) NOT NULL UNIQUE,
 password VARCHAR(100) NOT NULL, phone VARCHAR(15), designation VARCHAR(100) NOT NULL, dept_id INT NOT NULL,
 category ENUM('TEACHING','NON_TEACHING','ADMIN') NOT NULL,
 role ENUM('ADMIN','EMPLOYEE') NOT NULL DEFAULT 'EMPLOYEE', join_date DATE NOT NULL,
 basic_salary DECIMAL(10,2) NOT NULL CHECK (basic_salary > 0),
 status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
 FOREIGN KEY (dept_id) REFERENCES Departments(dept_id));
CREATE TABLE Attendance (
 att_id INT AUTO_INCREMENT PRIMARY KEY, emp_id INT NOT NULL, att_date DATE NOT NULL,
 check_in TIME NULL, check_out TIME NULL,
 status ENUM('PRESENT','LATE','ABSENT','ON_LEAVE') NOT NULL, UNIQUE (emp_id, att_date),
 FOREIGN KEY (emp_id) REFERENCES Employees(emp_id) ON DELETE CASCADE);
CREATE TABLE LeaveBalance (
 emp_id INT NOT NULL, leave_type ENUM('CL','ML','EL','OD') NOT NULL, balance INT NOT NULL CHECK (balance >= 0),
 PRIMARY KEY (emp_id, leave_type), FOREIGN KEY (emp_id) REFERENCES Employees(emp_id) ON DELETE CASCADE);
CREATE TABLE LeaveRequests (
 leave_id INT AUTO_INCREMENT PRIMARY KEY, emp_id INT NOT NULL, leave_type ENUM('CL','ML','EL','OD') NOT NULL,
 start_date DATE NOT NULL, end_date DATE NOT NULL, days INT NOT NULL, reason VARCHAR(255),
 status ENUM('PENDING','APPROVED','REJECTED','CANCELLED') NOT NULL DEFAULT 'PENDING',
 CHECK (end_date >= start_date), FOREIGN KEY (emp_id) REFERENCES Employees(emp_id) ON DELETE CASCADE);
CREATE TABLE Tasks (
 task_id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(150) NOT NULL, assigned_to INT NOT NULL,
 hours INT NOT NULL CHECK (hours > 0), due_date DATE NOT NULL,
 priority ENUM('LOW','MEDIUM','HIGH') NOT NULL DEFAULT 'MEDIUM',
 status ENUM('PENDING','IN_PROGRESS','COMPLETED') NOT NULL DEFAULT 'PENDING', completed_on DATE NULL,
 FOREIGN KEY (assigned_to) REFERENCES Employees(emp_id) ON DELETE CASCADE);
CREATE TABLE Payroll (
 pay_id INT AUTO_INCREMENT PRIMARY KEY, emp_id INT NOT NULL, pay_month INT NOT NULL CHECK (pay_month BETWEEN 1 AND 12),
 pay_year INT NOT NULL, basic DECIMAL(10,2) NOT NULL, hra DECIMAL(10,2) NOT NULL, da DECIMAL(10,2) NOT NULL,
 allowance DECIMAL(10,2) NOT NULL DEFAULT 0, pf DECIMAL(10,2) NOT NULL, lop_days INT NOT NULL DEFAULT 0,
 lop_amount DECIMAL(10,2) NOT NULL DEFAULT 0, net_salary DECIMAL(10,2) NOT NULL,
 UNIQUE (emp_id, pay_month, pay_year), FOREIGN KEY (emp_id) REFERENCES Employees(emp_id) ON DELETE CASCADE);
CREATE TABLE PerformanceReports (
 report_id INT AUTO_INCREMENT PRIMARY KEY, emp_id INT NOT NULL, period VARCHAR(20) NOT NULL,
 attendance_score DECIMAL(5,2) NOT NULL, task_score DECIMAL(5,2) NOT NULL, punctuality_score DECIMAL(5,2) NOT NULL,
 overall_score DECIMAL(5,2) NOT NULL, rating VARCHAR(30) NOT NULL, generated_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
 FOREIGN KEY (emp_id) REFERENCES Employees(emp_id) ON DELETE CASCADE);
INSERT INTO Departments (dept_name) VALUES
 ('Artificial Intelligence and Data Science'), ('Administration'), ('Library');
```

APPENDIX B
EXECUTION OUTPUT AND SCREENSHOTS

B.1 Complete console output of the logic verification (actual run)

```
=== 1. OOP and polymorphism ===
Arun Kumar -> TEACHING, allowance 4000.00, task limit 20 h
Meena Devi -> NON_TEACHING, allowance 0.00, task limit 30 h
[PASS] Teaching task limit  expected=20  actual=20
[PASS] Non-teaching task limit  expected=30  actual=30
[PASS] Admin role  expected=ADMIN  actual=ADMIN

=== 2. Payroll calculation ===
---------------------------------------------
PAYSLIP 10/2026
Employee : Arun Kumar (No. 2)
Basic : 50000.00
HRA (10%) : 5000.00
DA (12%) : 6000.00
Academic allow. : 4000.00
Gross salary : 65000.00
PF (12%) : 6000.00
LOP (2 day(s)) : 4333.33
NET SALARY : 54666.67
---------------------------------------------
[PASS] Teaching gross  expected=65000.00  actual=65000.00
[PASS] Teaching PF  expected=6000.00  actual=6000.00
[PASS] Teaching LOP (2 days)  expected=4333.33  actual=4333.33
[PASS] Teaching net  expected=54666.67  actual=54666.67
[PASS] Non-teaching gross  expected=36600.00  actual=36600.00
[PASS] Non-teaching net, no LOP  expected=33000.00  actual=33000.00
[PASS] Non-teaching LOP 1 day  expected=1220.00  actual=1220.00
[PASS] Non-teaching net, 1 LOP day  expected=31780.00  actual=31780.00

=== 3. AI performance score ===
[PASS] Overall A=92,T=80,P=90  expected=86.80  actual=86.80
[PASS] Attendance score (18,2,2,2)  expected=91.67  actual=91.67
[PASS] Task score 4 of 5  expected=80.00  actual=80.00
[PASS] Punctuality 18 present 2 late  expected=90.00  actual=90.00
[PASS] No attendance recorded  expected=0.00  actual=0.00
[PASS] No task due  expected=100.00  actual=100.00
[PASS] Rating 85  expected=Excellent  actual=Excellent
[PASS] Rating 84.99  expected=Good  actual=Good
[PASS] Rating 70  expected=Good  actual=Good
[PASS] Rating 69.99  expected=Average  actual=Average
[PASS] Rating 50  expected=Average  actual=Average
[PASS] Rating 49.99  expected=Needs Improvement  actual=Needs Improvement

=== 4. Workload analysis ===
[PASS] Teaching 24 h pending  expected=OVERLOADED  actual=OVERLOADED
[PASS] Non-teaching 15 h pending  expected=BALANCED  actual=BALANCED
[PASS] Teaching 6 h pending  expected=UNDERLOADED  actual=UNDERLOADED
[PASS] Exactly 100%  expected=BALANCED  actual=BALANCED
[PASS] Exactly 40%  expected=BALANCED  actual=BALANCED

=== 5. Attendance risk ===
[PASS] 2 late/absent  expected=LOW  actual=LOW
[PASS] 3 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 4 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 5 late/absent  expected=HIGH  actual=HIGH

=== 6. Task-assignment suggestion (greedy least-loaded) ===
Rank 1: Divya M  load after = 45.00%  score = 90.00
Rank 2: Karthik P  load after = 45.00%  score = 80.00
Rank 3: Ravi S  load after = 80.00%  score = 70.00
[PASS] Best candidate (tie by score)  expected=Divya M  actual=Divya M
[PASS] Last candidate  expected=Ravi S  actual=Ravi S

=== 7. Leave and attendance rules ===
[PASS] Days 5th to 5th  expected=1  actual=1
[PASS] Days 5th to 9th  expected=5  actual=5
[PASS] Check-in 08:50  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:15  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:16  expected=LATE  actual=LATE
[PASS] Overlap: 5-9 vs 8-12  expected=true  actual=true
[PASS] Overlap: 5-9 vs 10-12  expected=false  actual=false

Result: 41 passed, 0 failed.
```

B.2 Console screenshots of the verified logic (actual run)

```
● ● ●  java EmsLogic.java  -  payroll

=== 2. Payroll calculation ===
------------------------------------------
PAYSLIP 10/2026
Employee : Arun Kumar (No. 2)
Basic : 50000.00
HRA (10%) : 5000.00
DA (12%) : 6000.00
Academic allow. : 4000.00
Gross salary : 65000.00
PF (12%) : 6000.00
LOP (2 day(s)) : 4333.33
NET SALARY : 54666.67
------------------------------------------
[PASS] Teaching gross  expected=65000.00  actual=65000.00
[PASS] Teaching PF  expected=6000.00  actual=6000.00
[PASS] Teaching LOP (2 days)  expected=4333.33  actual=4333.33
[PASS] Teaching net  expected=54666.67  actual=54666.67
[PASS] Non-teaching gross  expected=36600.00  actual=36600.00
[PASS] Non-teaching net, no LOP  expected=33000.00  actual=33000.00
[PASS] Non-teaching LOP 1 day  expected=1220.00  actual=1220.00
[PASS] Non-teaching net, 1 LOP day  expected=31780.00  actual=31780.00

=== 3. AI performance score ===
```

Fig 10 Fig B1 Payroll payslip and payroll checks (java EmsLogic.java)

```
● ● ●  java EmsLogic.java  -  AI analytics

[PASS] Overall A=92,T=80,P=90  expected=86.80  actual=86.80
[PASS] Attendance score (18,2,2,2)  expected=91.67  actual=91.67
[PASS] Task score 4 of 5  expected=80.00  actual=80.00
[PASS] Punctuality 18 present 2 late  expected=90.00  actual=90.00
[PASS] No attendance recorded  expected=0.00  actual=0.00
[PASS] No task due  expected=100.00  actual=100.00
[PASS] Rating 85  expected=Excellent  actual=Excellent
[PASS] Rating 84.99  expected=Good  actual=Good
[PASS] Rating 70  expected=Good  actual=Good
[PASS] Rating 69.99  expected=Average  actual=Average
[PASS] Rating 50  expected=Average  actual=Average
[PASS] Rating 49.99  expected=Needs Improvement  actual=Needs Improvement

=== 4. Workload analysis ===
[PASS] Teaching 24 h pending  expected=OVERLOADED  actual=OVERLOADED
[PASS] Non-teaching 15 h pending  expected=BALANCED  actual=BALANCED
[PASS] Teaching 6 h pending  expected=UNDERLOADED  actual=UNDERLOADED
[PASS] Exactly 100%  expected=BALANCED  actual=BALANCED
[PASS] Exactly 40%  expected=BALANCED  actual=BALANCED

=== 5. Attendance risk ===
[PASS] 2 late/absent  expected=LOW  actual=LOW
[PASS] 3 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 4 late/absent  expected=MEDIUM  actual=MEDIUM
[PASS] 5 late/absent  expected=HIGH  actual=HIGH

=== 6. Task-assignment suggestion (greedy least-loaded) ===
Rank 1: Divya M  load after = 45.00%  score = 90.00
Rank 2: Karthik P  load after = 45.00%  score = 80.00
Rank 3: Ravi S  load after = 80.00%  score = 70.00
```

Fig 11 Fig B2 AI performance, workload and risk checks

```
● ● ●  java EmsLogic.java  -  summary

[PASS] Last candidate  expected=Ravi S  actual=Ravi S

=== 7. Leave and attendance rules ===
[PASS] Days 5th to 5th  expected=1  actual=1
[PASS] Days 5th to 9th  expected=5  actual=5
[PASS] Check-in 08:50  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:15  expected=PRESENT  actual=PRESENT
[PASS] Check-in 09:16  expected=LATE  actual=LATE
[PASS] Overlap: 5-9 vs 8-12  expected=true  actual=true
[PASS] Overlap: 5-9 vs 10-12  expected=false  actual=false

Result: 41 passed, 0 failed.
```

Fig 12 Fig B3 Leave/attendance rules and final result: 41 passed, 0 failed

B.3 Screenshots to be inserted after running with MySQL

Table 19 Screenshot checklist

Fig	Screen	Status
B4	Database connection and login	Insert screenshot
B5	Admin menu and employee list	Insert screenshot
B6	Attendance and leave approval	Insert screenshot
B7	Payroll report and payslip from MySQL	Insert screenshot
B8	MySQL Workbench: tables and sample rows	Insert screenshot

Screenshots are not generated artificially: they must be captured from the running application so that they match the code the team presents in the viva.

APPENDIX C
REQUIREMENT CHECKLIST

Table 20 Checklist

Requirement for this report	Where covered
Student Declaration page	Front matter
Chapters per new structure (Requirements and Feasibility, System Analysis and Design, Database Design, Implementation, Verification and Test Plan, Security/Limitations/Maintenance)	Chapters 2–7
Use case, DFD Level 0 and 1, ER, sequence (leave, payroll), class diagrams	Sections 3.1–3.6, 4.1
Normalisation 1NF–3NF, keys, constraints, ACID, indexes, views	Sections 4.2–4.6
CRUD mapping table	Section 4.7
Real code snippets: JDBC, PreparedStatement, transaction	Chapter 5
Test results and screenshots	Chapter 6 (logic verified); Appendix B (DB screenshots pending)
BCrypt plan	Section 7.3
AI future enhancements, global relevance, anti-proxy measures	Sections 1.5, 1.6, 8.2